Concepts

Ignacy Kos, Dawid Nogajski

Templates - revision

To understand concepts, it is necessary to know templates.

A template is a C++ entity that defines one of the following:

- a family of classes (class template), which may be nested classes
- a family of functions (function template), which may be member functions

In short, it is a class or a function which can accept different types as arguments.

Templates - example

```
template <typename T>
T add(T a, T b)
{
    return a + b;
}
```

```
int main()
{
    int x = 3;
    int y = 5;

    add(x, y);

    add<int>(6, 8);
    add<double>(6.0, 5.3);
}
```

Concepts

Concepts are compile-time constraints for templates added in C++20.

They define exactly what a type must be able to do before a template is instantiated, eliminating unreadable error messages and making code self-documenting.

They replace traditional, unconstrained generic types (typename T) with explicit requirements, allowing the compiler to reject invalid types instantly rather than failing deep within the template implementation.

Concept syntax

```
template <template-parameter-list>
concept ConceptName = constraint-expression;
```

The *`constraint-expression`* can either be a predefined concept (we are going to discuss them soon), multiple predefined concepts separated by “||” or “&&”, or a “requires” expression.

Requires expression

```
template <template-parameter-list>
concept ConceptName = requires(optional-parameter-list)
{ requires-sentence };
```

The requires expression makes it possible to define own concepts using a syntax similar to a function, but using a “requires” word instead of the function name.

Example of using a template without concepts

```cpp
#include <iostream>

template <typename T>
T add(T a, T b)
{
    return a + b;
}

class Example {};
```

```cpp
int main()
{
    Example ex1, ex2;

    add(ex1, ex2);
}
```

Example of using a template without concepts

Code	Description	Project	File	Line	Details
C2676	binary '+': 'T' does not define this operator or a conversion to a type acceptable to the predefined operator	test	test.cpp	13	

binary '+': 'T' does not define this operator or a conversion to a type acceptable to the predefined operator
with
[
T=Example
]

C2676 binary '+': 'T' does not define this operator or a conversion to a type acceptable to the predefined operator
with
[
T=Example
]
test.cpp (Line 13)

the template instantiation context (the oldest one first) is
test.cpp (Line 13)
see reference to function template instantiation 'T add<Example>(T,T)' being compiled
with
[
T=Example
]
test.cpp (Line 23)

Example of a concept

```
#include <iostream>
#include <concepts>

template <typename T>
concept Addable = requires(T a, T b)
{
    a + b;
};

template <Addable T>
T add(T a, T b)
{
    return a + b;
}

class Example {};
```

the name of the concept instead of "typename"

```
int main()
{
    Example ex1, ex2;

    add(ex1, ex2);
}
```

E0304 no instance of function template "add" matches the argument list
argument types are: (Example, Example)

Example of a concept

Code	Description	Project	File	Line	Details
E0304	no instance of function template "add" matches the argument list	test	test.cpp	23	
C2672	'add': no matching overloaded function found	test	test.cpp	23	

Problem Details

C2672 'add': no matching overloaded function found
test.cpp (Line 23)

- could be 'T add(T,T)' test.cpp (Line 11)
  - the associated constraints are not satisfied test.cpp (Line 23)
    - the concept 'Addable<Example>' evaluated to false test.cpp (Line 10)
      - binary '+': 'Example' does not define this operator or a conversion to a type acceptable to the predefined operator test.cpp (Line 7)

Clearer error messages!

Predefined concepts from the standard library

Apart from constructing own concepts, it is also possible to use concepts predefined in the standard library. To do that, it is necessary to include `<concepts>`. Here are some of the core concepts:

`std::same_as<T, U>`	specifies that a type is the same as another type
`std::convertible_to<T>` `std::convertible_to<From, To>`	specifies that a type is convertible to another type
`std::integral<T>`	specifies that a type is an integral type
`std::floating_point<T>`	specifies that a type is a floating-point type

Return type constraint

When creating concepts which return some value, it is possible to define a return type constraint. In other words, it is possible to specify what type should be returned.

```
template <typename T>
concept Addable = requires(T a, T b)
{
    { a + b } -> std::convertible_to<double>;
};
```

no semicolon here!

specifies the return type constraint - in this case that the value returned by a + b should be convertible to a double

you can also add a "`noexcept`" operator here (in front of the arrow); it checks whether the constraint evaluates to true during compilation

Nested requirements

We are not limited in how complex the requirements can be. To creating more specific concepts we need to explicitly use the ‘requires’ keyword inside the definition.

A standard concept simply checks for validity of syntax.

Adding another ‘requires’ forces the compiler to evaluate another independent condition at compile time.

Example of a nested requirement

```
template <typename T>
concept NetworkPacket = requires(T packet) {
    packet.serialize();

    requires sizeof(T) <= 256;

    requires std::is_trivially_copyable_v<T>;
};

void send_over_network(const NetworkPacket auto& packet) {
    std::cout << "Sending... << std::endl;
    std::cout << "Successfully sent " << sizeof(packet) << " bytes!\n";
}
```

Simple Requirement - we just check if this type has a method called serialize()

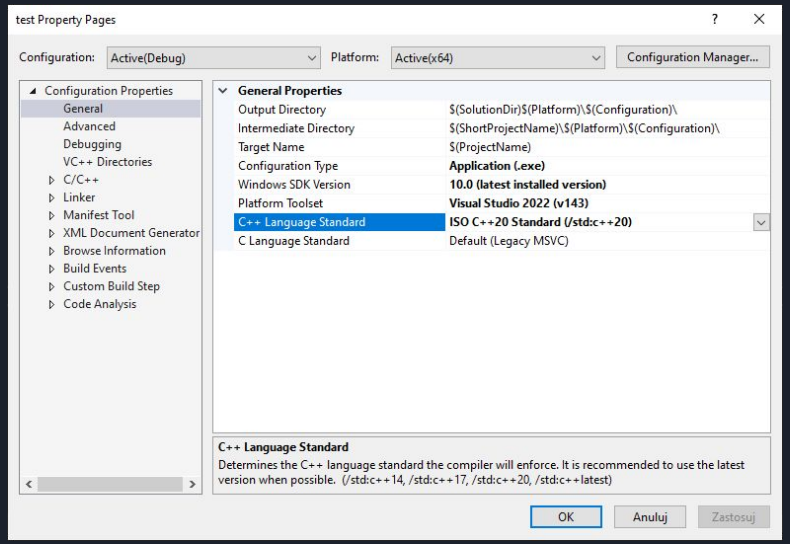
Nested Requirement - evaluates this at compile time: is the size of the struct 256 bytes or less?

Before you start working...

Remember that in order to use Concepts you need to change the C++ standard to C++20.

To do that in Visual Studio, from the top bar go to:

Project -> Properties -> Configuration Properties -> C++ Language Standard -> ISO C++20 Standard (/std:c++20)



Thank you for your attention

github.com/IgiBlackbird/cp4_concepts